

PAPSS Middleware Installer for Docker on Linux

1. Install Docker

Uninstall old versions

```
sudo yum remove docker \
    docker-client \
    docker-client-latest \
    docker-common \
    docker-latest \
    docker-latest-logrotate \
    docker-logrotate \
    docker-engine
```

Install Docker Engine using the rpm repository

Before you install Docker Engine for the first time on a new host machine, you need to set up the Docker repository. Afterward, you can install and update Docker from the repository.

Set up the repository

Install the yum-utils package (which provides the yum-config-manager utility) and set up the repository.

```
sudo yum install -y yum-utils
sudo yum-config-manager --add-repo
https://download.docker.com/linux/centos/docker-ce.repo
```

Install Docker engine, containerd and Docker compose

To install the latest version, run:

```
sudo yum install docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

If prompted to accept the GPG key, verify that the fingerprint matches **060A 61C5 1B55 8A7F 742B 77AA C52F EB6B 621E 9F35**, and if so, accept it.

This command installs Docker, but it doesn't start Docker. It also creates a docker group, however, it doesn't add any users to the group by default.

Start Docker.

```
sudo systemctl start docker
```

Verify that the Docker Engine installation is successful by running the hello-world image.

```
sudo docker run hello-world
```

This command downloads a test image and runs it in a container. When the container runs, it prints a confirmation message and exits.

You have now successfully installed and started Docker Engine.

For further instructions and troubleshooting follow the below official docker tutorial:

<https://docs.docker.com/engine/install/centos/>

Create PAPSS Middleware directory and cd to it

```
sudo mkdir -m777 -p /var/papss
sudo mkdir -m777 -p /var/papss/cert
sudo mkdir -m777 -p /var/papss/config
sudo mkdir -m777 -p /var/papss/db
cd /var/papss
```

2. Install Digital Certificate

For test environments, the papss team will share a Java Keystore file (JKS) `client.jks` with a digital certificate that PAPSS has already preconfigured for a given participant.

The `client.jks` file should be located on the path `/var/papss/cert`

For production environments, it is expected that the participant will purchase a digital certificate and with the help of the PAPSS team, create the jks file for use in production.

3. Update Configurations - application.yml

Configurations are located in a file named `application.yml` on the path `var/papss/config`

The following configurations are found in the `application.yml` file

1. `datasource configs` - database environment variables, supported databases are postgres and sql server.
2. `corebanking configs` - endpoints for participant corebanking system specifically for incoming messages
3. `papss participant configs` - configurations for the various tenant or subsidiaries that the participant might have, for example Kenya, Nigeria and Zambia subsidiaries of a given bank.
4. `ips configs` - client configurations to enable the middleware to connect to the central PAPSS IPS
5. `security configs` - security properties mainly for configuration of the Java Keystore (JKS) which contains the digital certificate
6. `system configs` - system configs for controlling the number of thread processes and how they startup.

1. datasource configs

SQL Server example configs

```
spring:
  dataSource:
    driverClassName: com.microsoft.sqlserver.jdbc.SQLServerDriver
    password: V3ry8igs3cr3t
    url:
jdbc:sqlserver://;serverName=0.0.0.0:1433;databaseName=papssdb;encrypt=true;trustServerCertificate=true;
    username: SA
    hikari:
      auto-commit: false
  jpa:
    hibernate:
      ddl-auto: update
    properties:
      hibernate:
        dialect: org.hibernate.dialect.SQLServerDialect
        format_sql: true
      show-sql: true
  sql:
    init:
      mode: always
```

Postgres SQL example configs

```
spring:
  dataSource:
    driverClassName: org.postgresql.Driver
    password: pap0cDB01d3v
    url: jdbc:postgresql://0.0.0.0:5432/papss_ips
    username: postgres
    hikari:
      auto-commit: false
  jpa:
    hibernate:
      ddl-auto: none
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
        format_sql: true
      show-sql: true
  sql:
    init:
      mode: always
```

Please copy the data source configs depending on which database you use (postgres or sqlserver).

Please ensure to change the `username`, `password` and `url` to suit your environment.

2. corebanking configs

```
core:
  banking:
    url: http://0.0.0.0:8889/papss-core-banking-mock-server
    path:
      credit-transfer: "/core-banking/credit-transfer"
      name-inquiry: "/core-banking/debtor/account-enquiry"
      recon: "/core-banking/settlement-recon"
      tx-confirmation: "/core-banking/tx-confirmation"
```

Please make sure to change the `url` to point to the core banking mock server as a start and eventually change this to point to the participant's core banking system after development of inbound APIs has been done.

You are also free to change the paths for the endpoints in your environment as you see fit.

3. papss participant configs

Example papss participant configs with 2 tenants configured

```
papss:
  participants:
    - apiKey: 706b3dac-21ca-4755-906f-841117db9eab
      country: LR
      inst_bic: AFXMLRLR
      inst_id: LR2020
      salt: supersecretsalt
      sslKeyAlias: afxm2
      keyStorePath: /var/papss/cert/client.jks
      keyPass: secure
      keyStoreType: JKS
      provider: SunJCE
    - apiKey: 706b3dac-21ca-4755-906f-841117db9eab
      country: GH
      inst_bic: PCMGGNCK
      inst_id: GH1004
      salt: supersecretsalt
      sslKeyAlias: tesb
      keyStorePath: /var/papss/cert/client.jks
      keyPass: secure
      keyStoreType: JKS
      provider: SunJCE
```

Please make sure to change these configurations to suit your papss participant configurations that have been shared with you by the PAPSS team.

4. ips configs

```
papss:
  ips:
    ipsDns: pm.test.papssnet.papss.com
    ipsPort: 443
    toClearingSystemId: XA0001
```

These configs are correct for test environments and dont need to be changed. Changes will have to be applied when migrating to production and the correct values will be shared by the PAPSS team.

5. security configs

```
papss:
  security:
    dSKeyFile: file:///var/papss/cert/client.jks
    dSKeyPath: /var/papss/cert/client.jks
    ipsDns: pm.test.papssnet.papss.com
    ipsPort: 443
    keyPass: secure
    keystoreType: JKS
    provider: SunJCE
    verifierProvider: SunJCE
```

The following configs need to be updated with the file path of jks file `dSKeyFile`, `dSKeyPath`. Please note that for test environment PAPSS will provide the `client.jks` file that contains the papss test digital certificates. In that regard, digital certificates will only need to be purchased when migrating to production environment.

6. system configs

```
papss:
  system:
    pollingExecutorThreadDelay: 10000
    pollingExecutorThreadRate: 360000
    pollingExecutorsPerParticipant: 3
    pollingSleepTime: 5000
    toClearingSystemId: XA0001
```

The `toClearingSystemId` will need to be changed for production environment, for the rest of the system configs the defaults work very well for most normal operational environments.

Full example of application.yml

```
spring:
  dataSource:
    driverClassName: com.microsoft.sqlserver.jdbc.SQLServerDriver
    password: V3ry8igs3cr3t
    url:
jdbc:sqlserver://;serverName=0.0.0.0:1433;databaseName=papssdb;encrypt=true;trustServerCertificate=true;
    username: SA
    hikari:
      auto-commit: false
  jpa:
    hibernate:
      ddl-auto: none
    properties:
      hibernate:
        dialect: org.hibernate.dialect.SQLServerDialect
        format_sql: true
      show-sql: true
  sql:
    init:
      mode: always
  logging:
    level:
      httpclient:
        wire: INFO
      org:
        apache:
          http: INFO
        springframework:
          web:
            client:
              RestTemplate: INFO
  core:
    banking:
      url: http://0.0.0.0:8889/papss-core-banking-mock-server
      path:
        credit-transfer: "/core-banking/credit-transfer"
        name-inquiry: "/core-banking/debtor/account-enquiry"
        recon: "/core-banking/settlement-recon"
        tx-confirmation: "/core-banking/tx-confirmation"
  papss:
    participants:
      - apiKey: 706b3dac-21ca-4755-906f-841117db9eab
        country: LR
        inst_bic: AFXMLRLR
        inst_id: LR2020
        salt: supersecretsalt
        sslKeyAlias: afxm2
        keyStorePath: /var/papss/cert/client.jks
```

```
    keyPass: secure
    keyStoreType: JKS
    provider: SunJCE
-   apiKey: 706b3dac-21ca-4755-906f-841117db9eab
    country: GH
    inst_bic: PCMGGNCK
    inst_id: GH1004
    salt: supersecretsalt
    sslKeyAlias: tesb
    keyStorePath: /var/papss/cert/client.jks
    keyPass: secure
    keyStoreType: JKS
    provider: SunJCE
ips:
  ipsDns: pm.test.papssnet.papss.com
  ipsPort: 443
  toClearingSystemId: XA0001
system:
  pollingExecutorThreadDelay: 10000
  pollingExecutorThreadRate: 360000
  pollingExecutorsPerParticipant: 3
  pollingSleepTime: 5000
  toClearingSystemId: XA0001
security:
  dSKeyFile: file:///var/papss/cert/client.jks
  dSKeyPath: /var/papss/cert/client.jks
  ipsDns: pm.test.papssnet.papss.com
  ipsPort: 443
  keyPass: secure
  keystoreType: JKS
  provider: SunJCE
  verifierProvider: SunJCE
springdoc:
  api-docs:
    enabled: true
    path: /rest-api-docs
  swagger-ui:
    path: /swagger-ui
```


4. Pull and Install Docker Containers

- 1. PostgreSQL Docker Container
- 2. PAPSS Inbound Service
- 3. PAPSS Outbound Service
- 4. PAPSS Core Banking Mock Server (TEST ENVIRONMENT ONLY)

1. Install PostgreSQL Container

Pull and run the postgresql docker image

```
docker run -itd -e POSTGRES_USER=papss -e POSTGRES_PASSWORD=s3cr3t -p 5432:5432 -v /var/papss/db:/var/lib/postgresql --name papss_postgresql postgres
```

SSH into the container to confirm setup

```
docker exec -it papss_postgresql bash
```

Connect to database using `psql` client

```
psql -h localhost -U papss
```

List databases to confirm `papss` databse has been created.

```
\l
```

Expected outcome:

```
papss=#
papss=# \l
```

List of databases									
Name	Owner	Encoding	Locale Provider	Collate	Ctype	ICU Locale	ICU Rules	Access privileges	
papss	papss	UTF8	libc	en_US.utf8	en_US.utf8				
postgres	papss	UTF8	libc	en_US.utf8	en_US.utf8				
template0	papss	UTF8	libc	en_US.utf8	en_US.utf8			=c/papss	+
template1	papss	UTF8	libc	en_US.utf8	en_US.utf8			=c/papss	+
								papss=CTc/papss	

```
(4 rows)
```

List users to confirm **papss** user is created with the correct roles and attributes.

```
\du
```

Expected outcome:

```
papss=# \du
                                List of roles
Role name | Attributes
-----+-----
papss    | Superuser, Create role, Create DB, Replication, Bypass RLS
papss=#
papss=#
papss=# █
```

The postgresQL DB is now successfully setup. For further instructions and troubleshooting follow the below tutorial

<https://medium.com/@basit26374/how-to-run-postgresql-in-docker-container-with-volume-bound-c141f94e4c5a>

2. Install PAPSS Outbound Service

Pull and run the **ghcr.io/papssafrica/papss-outbound-service:latest** container

```
docker run --detach --mount type=bind,src=/var/papss,target=/var/papss --
add-host=host.docker.internal:host-gateway -p 8881:8080
ghcr.io/papssafrica/papss-outbound-service:latest
```

3. Install PAPSS Inbound Service

Pull and run the **ghcr.io/papssafrica/papss-inbound-service:latest** container

```
docker run --detach --mount type=bind,src=/var/papss,target=/var/papss --
add-host=host.docker.internal:host-gateway -p 8882:8080
ghcr.io/papssafrica/papss-inbound-service:latest
```

4. Install PAPSS Core Banking Mock Server

Pull and run the **papss/papss-core-banking-mock-server:1.1** image

```
docker run --detach --name papss_core_banking_mock_server --add-
host=host.docker.internal:host-gateway -p 8889:8889 papss/papss-core-
banking-mock-server:1.1
```

Confirm Docker Containers are running

Expected outcome:

Confirm PAPSS Outbound Service is running by checking the docker container logs

Expected outcome from the logs:

11 / 14

Confirm PAPSS Inbound Service is running by checking the docker container logs

```
docker logs --follow CONTAINER_ID
```

This is achieved by checking the logs to confirm that the polling mechanish is running every 5 seconds

```

2024-04-08T13:41:42.469Z INFO 1 --- [main] c.c.c.ConfigServicePropertySourceLocator : Fetching config from server at : http://localhost:8888
2024-04-08T13:41:43.088Z INFO 1 --- [main] c.c.c.ConfigServicePropertySourceLocator : Exception on Url - http://localhost:8888/org.springframework.web.client.ResourceAccessException: I/O error on GET request for "http://localhost:8888/application/default": Connection refused. Will be trying the next url if available
2024-04-08T13:41:43.089Z WARN 1 --- [main] c.c.c.ConfigServicePropertySourceLocator : Could not locate PropertySource: I/O error on GET request for "http://localhost:8888/application/default": Connection refused
2024-04-08T13:41:43.162Z INFO 1 --- [main] c.p.i.PapssInboundIsoServiceApplication : No active profile set, falling back to 1 default profile: "default"
2024-04-08T13:41:50.034Z INFO 1 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2024-04-08T13:41:50.394Z INFO 1 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 297 ms. Found 1 JPA repository interface.
2024-04-08T13:41:52.162Z INFO 1 --- [main] o.s.cloud.context.scope.GenericScope : BeanFactory id=9aeb7603-c975-3705-be17-cb25f0f1616f
2024-04-08T13:41:53.058Z WARN 1 --- [main] trationDelegate$BeanPostProcessorChecker : Bean 'org.springframework.ws.config.annotation.DelegatingWsConfiguration' of type [org.springframework.ws.config.annotation.DelegatingWsConfiguration$$SpringCGI$$0] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying). The currently created BeanPostProcessor [annotationActionEndpointMapping] is declared through a non-static factory method on that class; consider declaring it as static instead.
2024-04-08T13:41:53.340Z INFO 1 --- [main] .w.s.a.s.AnnotationActionEndpointMapping : Supporting [WS-Addressing August 2004, WS-Addressing 1.0]
2024-04-08T13:41:58.033Z INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2024-04-08T13:41:58.098Z INFO 1 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2024-04-08T13:41:58.099Z INFO 1 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.17]
2024-04-08T13:41:58.380Z INFO 1 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2024-04-08T13:41:58.386Z INFO 1 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 15669 ms
2024-04-08T13:42:00.872Z INFO 1 --- [main] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [name: default]
2024-04-08T13:42:01.261Z INFO 1 --- [main] org.hibernate.Version : HHH000412: Hibernate ORM core version 6.4.1.Final
2024-04-08T13:42:01.501Z INFO 1 --- [main] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-level cache disabled
2024-04-08T13:42:03.672Z INFO 1 --- [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2024-04-08T13:42:03.888Z INFO 1 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2024-04-08T13:42:05.497Z INFO 1 --- [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection ConnectionID:1 ClientConnectionId: cd2b5c36-1afb-4b21-b8eb-3358d218bd88
2024-04-08T13:42:05.515Z INFO 1 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2024-04-08T13:42:05.717Z INFO 1 --- [main] org.hibernate.orm.deprecation : HHN9000025: SQLServerDialect does not need to be specified explicitly using 'hibernate.dialect' (remove the property setting and it will be selected by default)
2024-04-08T13:42:10.908Z INFO 1 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHN000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2024-04-08T13:42:10.940Z INFO 1 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2024-04-08T13:42:12.167Z INFO 1 --- [main] c.papss.inbound.security.KeyStoreLoader : Loading keystore from: /var/papss/cert/client.jks. Type: JKS
2024-04-08T13:42:13.148Z INFO 1 --- [main] c.papss.inbound.security.KeyStoreLoader : Loading keystore from: /var/papss/cert/client.jks. Type: JKS
2024-04-08T13:42:19.167Z WARN 1 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2024-04-08T13:42:25.733Z INFO 1 --- [main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 1 endpoint(s) beneath base path '/actuator'
2024-04-08T13:42:26.383Z INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path ''
2024-04-08T13:42:26.527Z INFO 1 --- [main] c.p.i.PapssInboundIsoServiceApplication : Started PapssInboundIsoServiceApplication in 55.47 seconds (process running for 59.976)
2024-04-08T13:42:26.789Z INFO 1 --- [main] c.p.i.polling.MessagePollingService : ===== Polling for PapssID:LR2020
2024-04-08T13:42:26.808Z INFO 1 --- [main] c.p.i.polling.MessagePollingService : ===== Polling for PapssID:GH1004

```

7. Testing

Testing can be achieved through 2 methods:

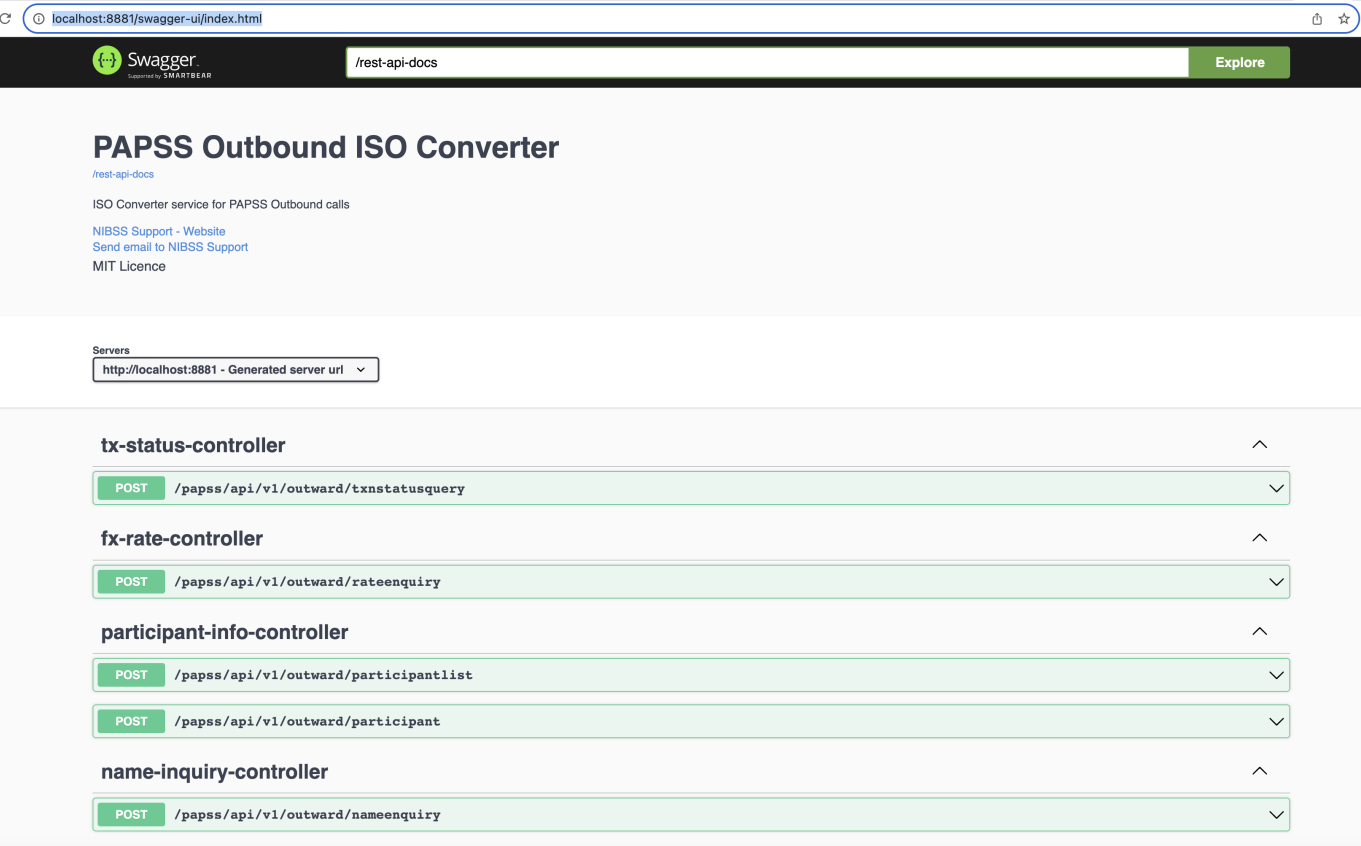
- 1. Testing with Swagger APIs
- 2. Testing with Postman Collection

1. Testing with Swagger APIs

The PAPSS-Outbound-ISO-Service has swagger endpoints available at:

<http://localhost:8881/swagger-ui/index.html>

Example Swagger UI:



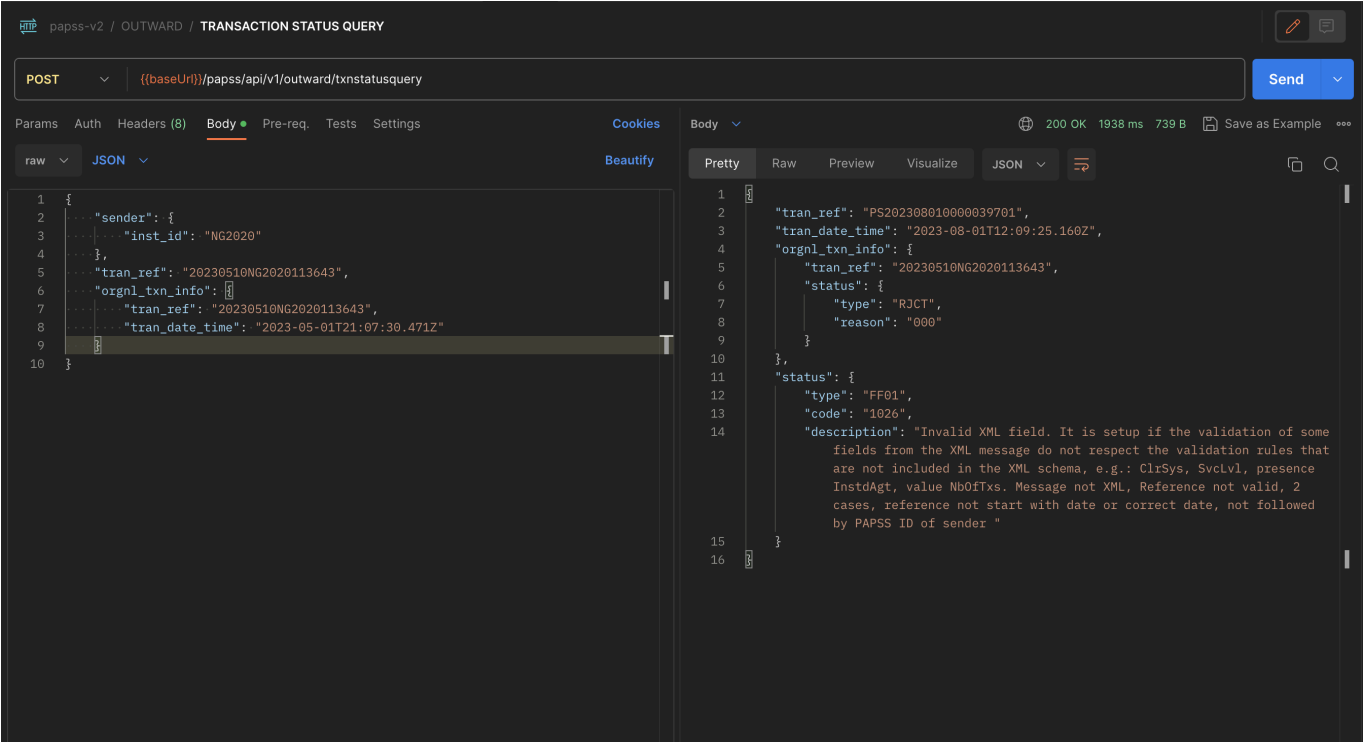
2. Testing with Postman Collection

A postman collection will be share by the PAPSS team for testing purposes.

The Postman collection `papss-v1.05.postman_collection.json` will be shared with you for testing.

Open this in Postman, change the `baseUrl` environment to suite your needs, and start testing.

Example Postman UI:



Make sure to updated the headers with the relevant values for `PAPSS-ID` and `PAPSS-API-KEY`.